AI Engineer

Take-Home Assignment

Build an Agent. Choose Your Challenge.

1. The Challenge

Build an AI-powered restaurant ordering assistant that gets to know its users over time.

Your assistant should help users discover and choose items from a restaurant menu based on their preferences, dietary restrictions, and previous interactions. But that is only the starting point.

This assignment consists of two parts:

Part 1 - The Baseline

Build a functional AI agent that meets a small set of core requirements.

Part 2 - Your Challenge

Once the baseline is working, identify an area you believe deserves deeper engineering investment, define your own scope, and take it further.

We are intentionally leaving the second part open-ended. We want to understand how you think as an engineer: what problems you identify, what you prioritize, the trade-offs you make, and how you turn your decisions into working software.

There is no single correct solution, architecture, or preferred area of specialization.

2. Part One - The Baseline

Build a conversational AI agent that helps users decide what to order from a restaurant. Your solution must support the following capabilities.

2.1 Understand the User

Your agent should engage in natural conversations to understand relevant user preferences, such as favorite foods and ingredients, dietary preferences and restrictions, disliked ingredients or dishes, budget considerations, and other preferences relevant to restaurant recommendations.

The agent should gather this information naturally through conversations rather than requiring users to complete a predefined questionnaire.

2.2 Recommend Menu Items

Your agent should recommend suitable items from an available restaurant menu. Recommendations should consider the user's expressed preferences, restrictions, and current request.

All recommendations must be grounded in actual menu data. The agent should not invent dishes, ingredients, prices, or other menu information.

2.3 Long-Term Memory

Your agent must maintain persistent memory about users across independent conversations. For example, if a user mentions a preference during one conversation, the agent should be able to remember and use that information in a future conversation without requiring the user to repeat it.

Memory should meaningfully influence the agent's recommendations.

2.4 Evolving Preferences

User preferences are not necessarily permanent. Your agent should support situations where users correct, update, or change previously expressed preferences. The agent should adapt its understanding accordingly rather than blindly relying on outdated information.

2.5 Returning Users

The application must support identifying returning users so their previous preferences can be retrieved across separate conversations. You are free to determine how user identity is managed, provided the functionality can be demonstrated and tested.

Different users' preferences and memories should remain isolated.

2.6 Usable Interface

Your agent must be accessible through a deployed application that our team can interact with. The interface and interaction model are entirely up to you. A simple chat interface is sufficient for the baseline. You are not expected to build a production-grade frontend.

3. Restaurant Menu

You are free to design your own restaurant and menu. Your menu should contain enough variety and information to meaningfully test the agent's ability to understand preferences, reason about constraints, and recommend suitable items.

Minimum Requirements

- At least **12 distinct items** across **3 categories**.
- Items with different prices.
- Vegetarian and non-vegetarian options.
- Different spice levels.
- Explicit ingredient and allergen information.
- Multiple items with overlapping characteristics, allowing meaningful comparisons and recommendations.

Design your menu to support situations involving multiple preferences, dietary restrictions, budget constraints, and potentially conflicting requirements.

Implementation Freedom

Your menu may be stored in a local JSON file, stored in a database, retrieved through an API, or integrated with an external restaurant or menu service. The choice is yours.

You are **not required to build a restaurant management system or a complex integration layer** as part of the baseline. If you choose restaurant integrations as your deep-dive challenge, you are welcome to extend the menu model and supporting infrastructure.

4. Part Two - Choose Your Challenge

This is the most important part of the assignment.

Once your baseline is functional, choose one area of your solution that you believe deserves deeper engineering investment. You decide what to work on.

We are deliberately not prescribing the problem, architecture, or expected solution. We want to see what you identify as worth improving and how you approach solving it.

Choose Your Challenge (continued)

Possible Directions

The following are examples to inspire you, not a restricted list.

Memory & Personalization

Explore how the agent stores, retrieves, updates, or reasons about user preferences. You might investigate memory relevance, conflicting information, personalization quality, or long-term memory management.

Restaurant Integrations

Design an extensible integration layer that allows the agent to work with different restaurant systems, menus, or external APIs. You might explore abstraction boundaries, integration reliability, data consistency, or extensibility.

Scalability & Performance

Investigate how your solution behaves under increased workloads. You might explore concurrency, latency, resource utilization, architectural bottlenecks, or system performance.

User Experience

Rethink how users interact with the assistant. You might explore contextual recommendations, alternative interaction patterns, proactive assistance, or a more intuitive ordering experience.

Voice Interaction

Extend your agent to support real-time voice conversations. You might explore speech processing, latency, interruption handling, or the overall conversational experience.

Reliability & Evaluation

Investigate how to make your agent more dependable. You might explore automated evaluations, failure handling, recommendation accuracy, observability, or recovery mechanisms.

Something Else Entirely

You are not limited to these examples. If you identify a different problem that you believe is more interesting or valuable, pursue it.

Your Decisions & Technical Freedom

What We Want to Understand

Your chosen challenge should demonstrate four things:

- **Problem Identification:** What problem or opportunity did you identify? Why is it meaningful?
- **Prioritization:** Why did you choose this challenge over other possible improvements? What makes it worth investing in?
- **Scoping & Trade-offs:** What specifically did you decide to build? What did you deliberately leave out? What trade-offs did you make, and why?
- **Execution & Validation:** How did you approach the problem? What did you implement? How can you demonstrate that your solution works or improves the system?

A Note on Complexity

We do not reward complexity for its own sake. A thoughtfully designed, well-executed improvement is more valuable than an unnecessarily complicated architecture. You are not expected to solve every possible problem.

Choosing what not to build is just as important as choosing what to build.

5. Technical Freedom

You have complete freedom to choose your technical stack. This includes:

- Programming languages and frameworks.
- LLM providers and models.
- Agent frameworks or custom implementations.
- Databases and memory systems.
- Architecture and infrastructure.
- Frontend technologies.
- Deployment platforms.
- Testing and evaluation approaches.

You are welcome to use AI coding assistants and other development tools. We care about your ability to make appropriate engineering decisions, not whether you write every line of code manually.

However, you are responsible for understanding, validating, and explaining your implementation. You should be comfortable discussing any part of the solution you submit.

6. Deliverables

Your submission must include the following.

6.1 Deployed Application

Provide a working URL where our team can interact with your agent. The application must:

- Be accessible to our reviewers.
- Support testing the baseline functionality.
- Include your chosen deep-dive improvement.
- Remain available throughout the evaluation process.

Provide any necessary test credentials or instructions. A production-grade deployment is not required, but the application must be functional and usable.

6.2 Source Code

Provide access to your source code through a repository. Include a README covering how to run the project locally, an overview of the architecture, key technical decisions, important setup requirements, and known limitations.

Do not commit API keys, credentials, or other secrets.

6.3 Video Walkthrough

Record a video presenting your solution. **Recommended duration: 8-12 minutes.** Your video should cover:

- **Baseline Demonstration:** Demonstrate conversational recommendations, long-term memory across independent conversations, personalized recommendations based on remembered preferences, and updating previously learned preferences.
- **Your Chosen Challenge:** Explain what challenge you selected, why you selected it, and what problem you were trying to solve.
- **Technical Deep Dive:** Walk through the relevant parts of your implementation. Explain the architecture, technical decisions, and interesting engineering challenges you encountered.
- **Scope & Trade-offs:** Discuss what you prioritized, what you intentionally excluded, alternatives you considered, and what you would approach differently with more time.
- **Results & Validation:** Demonstrate the outcome of your chosen challenge. Explain how you tested or validated your solution and what evidence supports its effectiveness.

We are more interested in understanding your engineering decisions than watching a polished presentation.

Submission & Timeline

6.4 Submission

Upload your video walkthrough and any supporting materials to a Google Drive folder. Share the folder with the hiring team using the submission instructions provided by HR.

Your submission must include:

- Deployed application URL.
- Source code repository URL.
- Video walkthrough.
- Any necessary access instructions.

Please verify that all links and permissions work before submitting.

7. Timeline

You will have **72 consecutive hours** to complete and submit the assignment.

When Does the Timer Start?

Once you confirm your readiness to HR, the assignment will be sent to you. Your 72-hour window begins when HR sends the assignment. The exact submission deadline will be communicated by HR.

Expected Effort

We recommend allocating approximately **8-12 hours of focused work** within the 72-hour window. You are free to manage your time as you see fit.

The assignment is intentionally designed to evaluate how you prioritize and scope work under constraints. We do not expect a production-grade system or an exhaustive feature set.

Focus on delivering a functional baseline and a meaningful, well-executed improvement.

8. How We Evaluate Your Work

We are interested in more than whether your solution works. We want to understand how you approach engineering problems, make technical decisions, prioritize opportunities, and manage trade-offs.

There is no single correct architecture or preferred area of specialization. We encourage you to choose a challenge that you find meaningful and demonstrate your engineering strengths through your approach.

We value:

- Working software over theoretical designs.
- Engineering judgment over unnecessary complexity.
- Thoughtful scoping over excessive feature development.
- Demonstrated results over unsupported claims.
- Clear reasoning and ownership of technical decisions.

Technical Interview

Your submission will form the foundation of your subsequent technical interview. The interview will be driven primarily by what you built, the challenge you chose, and the decisions you made along the way.

We will explore your implementation, architecture, trade-offs, technical depth, and understanding of the problems you attempted to solve.

There is no expectation that your solution will be perfect. We are interested in understanding how you think, how you approach problems, and how you learn from your engineering decisions. Be prepared to discuss and defend your choices.

9. Final Notes

This assignment is intentionally open-ended. We are not looking for everyone to build the same solution. We want to see your approach to engineering.

Start by establishing a working baseline. Then identify something you believe is worth improving. Choose your challenge, define its scope, and invest your effort where you believe it matters most.

You have limited time, and how you use that time is part of the exercise.

Build the baseline. Find the opportunity. Make it better. Show us why your approach works.

Good luck!